深圳超算二期用户手册

一、集群简介 1
1.术语列表 1
2.计算节点配置 1
3.操作系统 1
4.作业调度系统 1
二、前期准备 2
1.报备 mac 地址 2
2.全盘扫描 3
3.接入集群 3
4.节点间免密配置 3
三、 集群环境 4
1.登录集群 4
2.HPCKit 4
3. 应用软件 4
使用方式 4
应用列表 4
4.python 离线包安装流程（以 numpy 为例） 7
5.公共工具 9
四、 作业提交 11
1.交互式提交方式 11
2.作业脚本提交 11
3.作业任务运行方式 14
五、数据传输 19
六、Web 页面登录使用 19
七、集群基础命令 22
1.状态查询&作业管理 22
2.调度脚本常用参数示例 23

一、集群简介

1.术语列表

术语名	解释
灵晟	深圳超算 E 级超级计算机系统
多瑙（Donau）	集群调度系统，提供作业提交、队列管理、资源分配等功能。
Donau Portal	多瑙调度器的图形界面，为用户访问集群资源提供方便易用的入口。用户使用用户名和密码登录 Donau Portal，成功授权后获取系统访问权限。
登录节点	用户接入集群的入口，用于代码编译、作业提交和轻量级文件操作。严禁在此节点运行计算任务。
计算节点	实际执行计算任务的服务器，通过调度系统统一分配资源。
队列	队列是作业被调度器调度时的通道，作业只有被提交到某个队列后才可以被调度器调度。

2.计算节点配置

每个灵晟计算节点包含 2 个 304 核、ArmV8 架构的 CPU（每个物理核为单线程），CPU 主频 1.55Ghz，内存每个 CPU 具备 32GB HBM，可访问 512GB DDR5 内存，节点间通过 200Gb Roce 网络连接。

3.操作系统

灵晟超级计算机使用银河麒麟高级服务器操作系统 "Kylin Linux Advanced Server V10 (Jasmine)"；内核版本 5.10.0-285.2.v2403.ky10p.aarch64。

4. 作业调度系统

灵晟超级计算机使用国产自研调度器多瑙 Donau ，负责整个集群资源管理和作业管理。用户将作业提交给调度器排队运行，调度器根据作业的资源需求和调度策略，从集群中分配合理的计算资源运行作业。

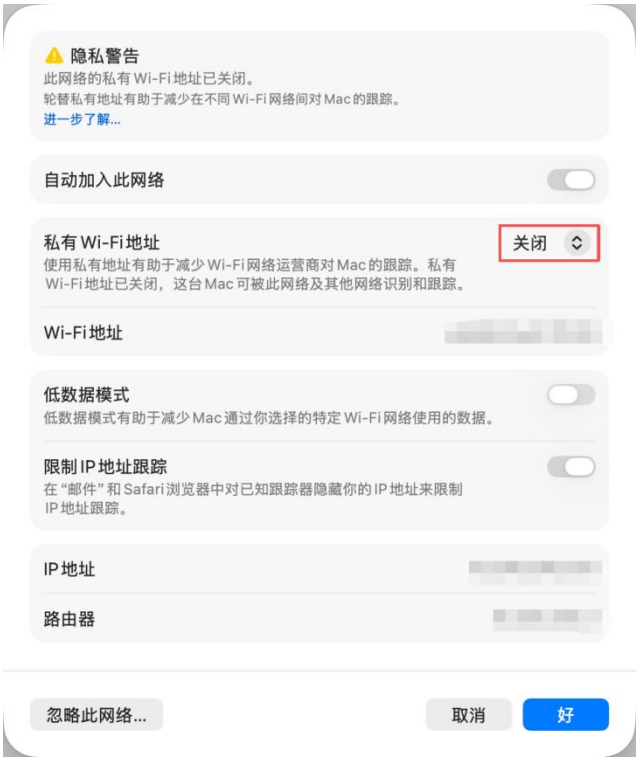
二、前期准备

1.报备 mac 地址

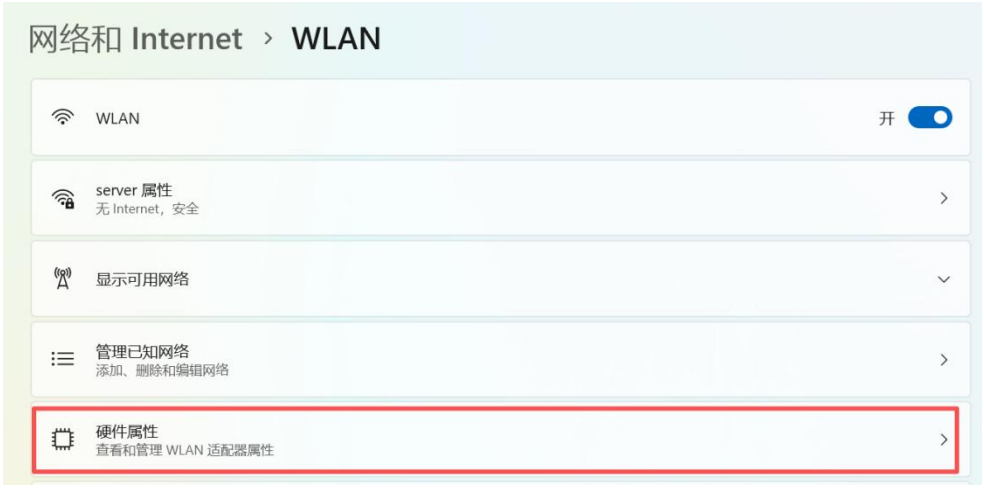
先检查并关闭“随机硬件地址”功能。

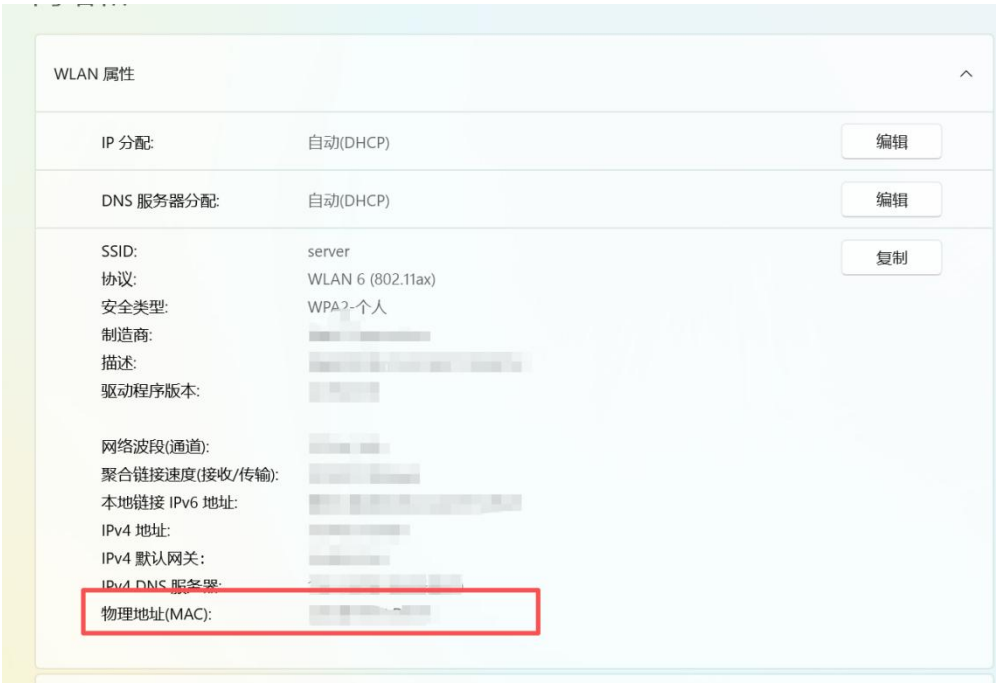


对于 mac 电脑：随意连接一个 wifi 后打开详情，关闭私有 wifi 地址，即可获得 MAC 地址



对于 windows 电脑：在网络和 Internet 中选择 WLAN，点开硬件属性即可看到 MAC 地址。





获得 MAC 地址之后报备给现场工作人员。

2.全盘扫描

提供任意杀毒软件全盘扫描的截图给现场工作人员(全盘杀毒扫描时间较久，建议提前准备)。

3.接入集群

可以连接 server wifi 即认为可以成功接入集群，现场工作人员会给你开通集群账号。

使用任意 ssh 工具连接登录节点 10.39.0.[1-40]，输入开通的集群账号密码即可接入集群。

4.节点间免密配置

```
cd ~
mkdir -p .ssh
```

```
echo "StrictHostKeyChecking no" > .ssh/config
echo -e "y\n" | ssh-keygen -t rsa -f ~/.ssh/id_rsa -N ""
cat .ssh/id_rsa.pub > .ssh/authorized_keys
chmod 600 .ssh/config
chmod 600 .ssh/authorized_keys
chmod 700 .ssh
chmod 700 $HOME
```

三、集群环境

1. 登录集群

连接到登录节点，执行命令 dlogin，输入账号密码，完成登录认证授权，使用命令 dqueue 查看当前账号可用队列。

```
[test1@login01 ~]$ dsub
dsub        dsubForNet
[test1@login01 ~]$ dlogin
Please enter the password:
**********************************************************************
The token is obtained successfully.
[test1@login01 ~]$ dqueue
NAME                        STATUS          PRIORITY   RUNNING_JOBS    PENDING_JOBS    SSTOPPED_JOBS
default                     OPEN,ACTIVE     1          0               0               0
q_app                       OPEN,ACTIVE     1          3               0               0
```

2. HPCKit

包含了编译器 clang、clang++、flang，并行库 hmpi，数学库 kml 等常用软件环境

```
source /work_ssd/software/HPCKit/25.2.1/setvars.sh
```

3. 应用软件

使用方式

集群已内置部分应用和工具/库，可以通过如下方式加载后使用

module use /work_ssd/software/modulefile/modules/

module load <app_name>/<app_version>-<hpckit_version>-<compiler_version>

应用列表

分类	应用名	版本
地震学	specfem3d	4.1.1
第一性原理	QE	v7.3.1
分子动力学	DeePMD-kit	3.0.1

分类	应用名	版本
分子动力学	Lammps	2023.8.2
分子动力学	gromacs	2024.2
流体力学	OpenFOAM	V2412
气象海洋	WRF	4.6
生命科学	Relion	4.0.1
生命科学	AlphaFold2	2
分子动力学	SPONGE	1.4
生命科学	AlphaFold3	3.0.1
大语言模型	deepseek	R1
第一性原理	CP2K	7.1
分子动力学	NAMD	3.0b6
流体力学	PALABOS	2.1r0
气象海洋	CESM	1.2.2
气象海洋	NEMO	4.2.2
气象海洋	WAVEWATCH	6.07.1
气象海洋	CAMx	7.31
生命科学	BWA	0.7.17
生命科学	Augustus	3.3.3
生命科学	GATK	4.0.0.0
生命科学	Chaste	2019.1
生命科学	phonopy	2.25.0
材料科学	wannier tools	2.7.1
生命科学	phono3py	3.2.0
气象海洋	CESM	2.2.2
气象海洋	NCL	6.3.0
深度学习	xla	tensorflow2.13.0
深度学习	tvm	0.16.0

分类	应用名	版本
基础科研	TensorFlow	2.13.0
基础科研	gulp	6.2
气象海洋	CDO	1.9.8
基础科研	Multiwfn	3.8(dev)
气象海洋	NCO	5.2.6
基础科研	Wannier90	3.1.0
气象海洋	ncview	2.1.5
基础科研	Anaconda3	2023.03
基础科研	SCALAPACK	2.0.2
基础科研	OpenBLAS	0.3.18
基础科研	boost	1.72
基础科研	CMAKE	3.26.0
基础科研	BLAS	3.8.0
气象海洋	gsl	2.6
基础科研	FFTW	3.3.8
基础科研	Lapack	3.8.0
深度学习	PyTorch	2.9.0
基础科研	KML	2.3.0
基础科研	bishengcompiler	4.0.0
基础科研	bishengJDK	11.0.23+11
基础科研	HMPI	2.3.0

详细列表可在 module use /work_ssd/software/modulefile/module 之后，通过 grep 查找需要的应用

以 vasp 为例，由于 module list 很长，建议使用 grep 的方式查找你需要的软件

```
module use /work_ssd/software/modulefile/modules
module av | grep vasp
module load vasp/6.3.2-hpckit25.1.0.SPC001-bisheng4.2.0.2.B002-
hmpi25.1.0.SPC001
```

4. python 离线包安装流程（以 numpy 为例）

①.准备 miniconda

下载地址：
https://blog.csdn.net/ming12131342/article/details/140233867

Miniconda3-latest-Linux-aarch64.sh	上传集群	149.2 MiB	2025-12-17 05:40
Miniconda3-latest-Linux-armv7l.sh		29.9 MiB	2025-02-14 03:43
Miniconda3-latest-Linux-ppc64le.sh		94.9 MiB	2025-02-14 03:43
Miniconda3-latest-Linux-s390x.sh		143.4 MiB	2025-02-14 03:43
Miniconda3-latest-Linux-x86.sh		62.7 MiB	2025-02-14 03:43
Miniconda3-latest-Linux-x86_64.sh		149.5 MiB	2025-12-17 05:40
Miniconda3-latest-MacOSX-arm64.pkg		115.4 MiB	2025-12-17 05:40
Miniconda3-latest-MacOSX-arm64.sh		116.1 MiB	2025-12-17 05:40
Miniconda3-latest-MacOSX-x86.sh		26.0 MiB	2025-02-14 03:43
Miniconda3-latest-MacOSX-x86_64.pkg		112.3 MiB	2025-08-26 05:03
Miniconda3-latest-MacOSX-x86_64.sh		112.9 MiB	2025-08-26 05:03
Miniconda3-latest-Windows-x86.exe		67.8 MiB	2025-02-14 03:43
Miniconda3-latest-Windows-x86_64.exe	本地使用	90.0 MiB	2025-12-17 05:40

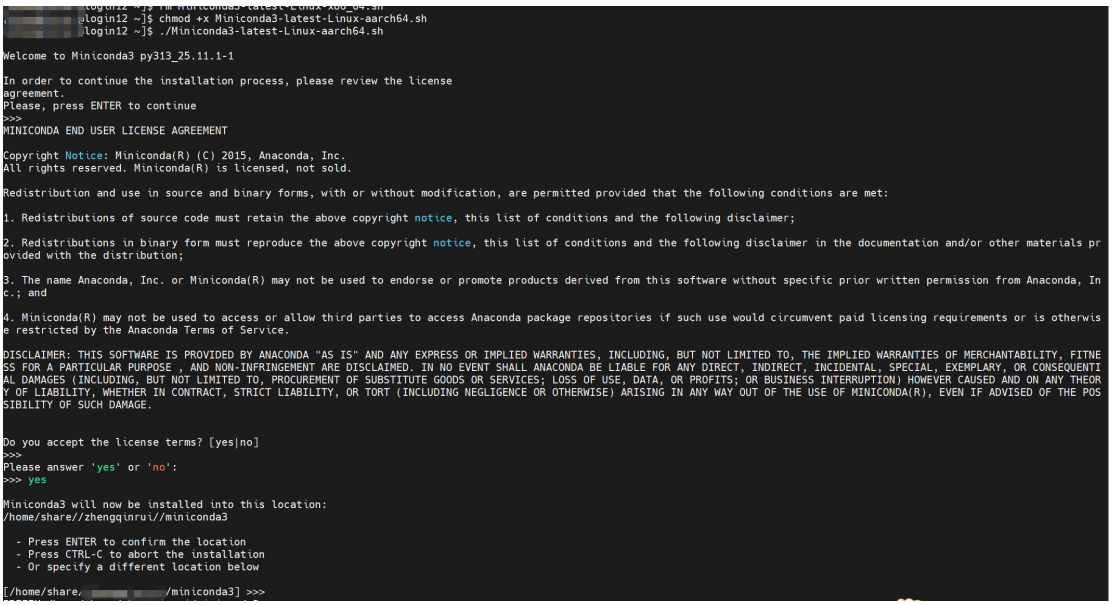
（建议：使用 latest 会默认 python3.13，如果不需要这个版本可以找到对应版本的安装包下载）

本地通过执行 exe，按图形界面指引进行安装。

集群端可以在上传文件后通过执行

```
Bash
Bash
chmod +x Miniconda3-latest-Linux-aarch64.sh
./Miniconda3-latest-Linux-aarch64.sh
```

进行安装



如果不需要使用复杂环境，也可以默认加载 miniconda，以下选择时选择 yes 即可。

```
Downloading and Extracting Packages:

Preparing transaction: done
Executing transaction: done
installation finished.
Do you wish to update your shell profile to automatically initialize conda?
This will activate conda on startup and change the command prompt when activated.
If you'd prefer that conda's base environment not be activated on startup,
   run the following command when conda is activated:

conda config --set auto_activate_base false

Note: You can undo this later by running `conda init --reverse $SHELL`

Proceed with initialization? [yes|no]
[no] >>> no

You have chosen to not have conda modify your shell scripts at all.
To activate conda's base environment in your current shell session:

eval "$(/home/share/zhengqinrui/miniconda3/bin/conda shell.YOUR_SHELL_NAME hook)"

To install conda's shell functions for easier access, first activate, then:

conda init

Thank you for installing Miniconda3!
```

②.下载 wheel 文件

本地下载命令:

```
Bash
Bash
pip download numpy -i https://pypi.tuna.tsinghua.edu.cn/simple --
only-binary=:all:  --platform manylinux_2_27_aarch64 --platform
manylinux2014_aarch64  --python-version 313
```

其中 numpy 可以替换成你需要的其他包，python-version 后面的数字根据你之前下载 miniconda 时选择的 python 版本调整，其它参数请不要调整

```
C:\Users\PC>pip download numpy  -i https://pypi.tuna.tsinghua.edu.cn/simple --only-binary=:all:  --platform manylinux_2_27_aarch64 --platform manylinux2014_aarch64  --python-vers
ion 313
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting numpy
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/8c/a4/77333f4d1e4dac4395385482557aeecf4826e6ff517e32ca48e1dafbe42a/numpy-2.4.1-cp313-cp313-manylinux_2_27_aarch64.manylinu
x_2_28_aarch64.whl (14.4 MB)
     ---------------------------------------- 14.4/14.4 MB 2.7 MB/s  0:00:05
Saved c:\users\pc\numpy-2.4.1-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl
Successfully downloaded numpy
```

注意到图中 Saved c:\users\pc\numpy-2.4.1-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl

将此 whl 文件上传至集群。

③.集群安装

在集群端你上传 whl 的目录下执行

```
Bash
Bash
source miniconda3/etc/profile.d/conda.sh
conda activate base
pip install --no-index --find-links=./ *.whl
```

输出以下内容可以认为安装成功：

```
(base) [redacted]@login12 ~]$ pip install --no-index --find-links=./ *.whl
Looking in links: ./
Processing ./numpy-2.4.1-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl
Installing collected packages: numpy
Successfully installed numpy-2.4.1
(base) [redacted]@login12 ~]$ python
Python 3.13.11 | packaged by Anaconda, Inc. | (main, Dec 10 2025, 21:29:12) [GCC 14.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>>
```

5. 公共工具

huge page

使用方式

方式一：交互式手动设置

提交交互式作业（示例节点为 cn22812）

```
dsub -q q_app_compile -nl "cn22812" -nn 2 -rpn 1 -I bash
```

进入工具目录

```
cd /work_ssd/software/performance_tune
```

运行大页脚本

```
./root_proxy hugepage.sh 0 8192 0 0 0 0 0 0
```

方式二：作业提交方式设置

```
#!/bin/bash
#DSUB -n hugepage
#DSUB --mpi hmpi
#DSUB -q q_app_compile
#DSUB -nl "cn[22812-22813]"
#DSUB -nn 2
#DSUB -rpn 32
#DSUB -x job
#DSUB -o n1_out_%J.log
#DSUB -e n1_err_%J.log

### 以上为作业头，分配节点 2 个， 每个节点 32 个资源副本，合计 64 个资源分布，且每节点独占
WORK_DIR=$(pwd)
{
  set  -x
  ### 进行节点的大页设置
  ### 步骤 1：进入脚本目录
  cd /work_ssd/software/performance_tune
```

步骤 2：设置大页(8129 * 2M)，把需要配置的节点都加入 drun 的-nl 列表。此示例中使用 cn22812，cn22813 节点

```
  export MASK_NODE=$(( 2#1111111111111110))
  drun -nl "cn[22812-22813]" -N 2 -rpn 1 ./root_proxy hugepage.sh 0 8192 0 0 0 0 0 0 ${MASK_NODE}
  set +x
}
cd ${WORK_DIR}
### MPI 程序主体，可以透明使用透明大页
{
  source env.sh
  mpirun -np $((32 * 2 )) \
      --mca coll ^ucg -mca pml ucx -mca btl ^vader,tcp,openib,uct -mca io romio321 \
      --bind-to core --map-by socket --rank-by core -map-by ppr:32:node:pe=1  \
      -x UCX_TLS=sm,rc -x PATH -x LD_LIBRARY_PATH -x UCX_NET_DEVICES=roceroh0:1  HelloWorld
}
```

关闭：

```
{
  ### 步骤 1：进入脚本目录
  cd /work_ssd/software/performance_tune
  ### 步骤 2：回收大页内存
  export MASK_NODE=$(( 2#1111111111111111))
  drun -nl "cn[22812-22813]" -N 2 -rpn 1 ./root_proxy hugepage.sh 0 0 0 0 0 0 0 0 ${MASK_NODE}
}
```

参数说明

```
Usage: ./root_proxy hugepage.sh <ddr_64k> <ddr_2m> <ddr_32m> <ddr_1g> <hbm_64k> <hbm_2m> <hbm_32m> <hbm_1g> NODE_MASK [quiet]
Example: ./root_proxy hugepage.sh 16 8 2 4 8 4 1 2 NODE_MASK quiet
# quiet: any non-empty value suppresses all output
```

drop cache

方式一：交互式手动设置

提交交互式作业（示例节点为 cn22812）

dsub -q q_app_compile -nl "cn22812" -nn 2 -rpn 1 -I bash

进入工具目录

cd /work_ssd/software/performance_tune/

运行脚本

./root_proxy drop_cache.sh

方式二：作业脚本中设置

在作业脚本中添加如下代码

cd /work_ssd/software/performance_tune

drun -nl "cn[22812-22813]" -N 2 -rpn 1 ./root_proxy hugepage.sh 0 0 0 0 0 0 0 0 ${MASK_NODE}

四、作业提交

1. 交互式提交方式

交互式作业，指作业提交后不立即退出，CLI 会持续等待并显示部分作业的状态变化及实时输出。当作业需要用户输入时，用户可在当前 CLI session 输入，然后作业继续运行直到结束，该作业在 CLI session 断开后，会被强制结束。

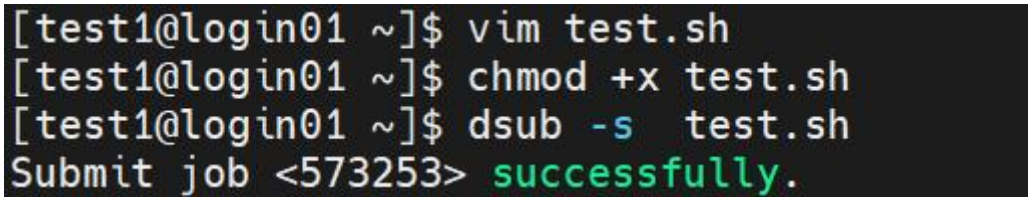
命令格式

```
dsub -I -q {your_queue}  bash
```

your_queue 为当前账号可用队列。

2. 作业脚本提交

编写作业脚本文件，修改节点数、进程数、线程数等需求，不需要改变 rankfile 生成和 mpirun 运行参数，添加可执行权限，使用命令 dsub -s 脚本提交作业



查看输出文件 output

```
[test1@login01 ~]$ cat test.sh
#!/bin/bash
#DSUB -n test
#DSUB -N 4
#DSUB -q q_app
#DSUB -o output
date
echo "this is script job"
[test1@login01 ~]$ cat output
Thu Apr  2 03:17:32 PM CST 2026
this is script job
```

使用 dattach 命令可以连接到作业执行节点（仅支持连接 RUNNING 状态作业），使用 exit 命令退出。单节点作业使用命令 dattach jobid 连接到作业执行节点。

```
[test1@login01 ~]$ dsub -q q_app -nl cn00140 sleep 100
Submit job <573277> successfully.
[test1@login01 ~]$ dattach 573277
<<<attaching to the node cn00140 of job 573277>>>
[test1@cn00140 ~]$ exit
exit
[test1@login01 ~]$
```

多节点作业使用命令 dattach jobid 默认连接到第一个节点，使用命令 dattach -en "node" jobid 连接到指定节点。

```
[test1@login01 ~]$ dsub -N 4 -q q_app  -nn 2 -nl "cn00139,cn00140" sleep 100
Submit job <573303> successfully.
[test1@login01 ~]$ dattach 573303
<<<attaching to the node cn00139 of job 573303>>>
[test1@cn00139 ~]$ exit
exit
[test1@login01 ~]$ dattach -en "cn00140" 573303
<<<attaching to the node cn00140 of job 573303>>>
[test1@cn00140 ~]$
```

普通作业脚本内容参考：

```
#!/bin/bash
#DSUB -n test
#DSUB -N 4
#DSUB -q q_app
#DSUB -o output
date
echo "this is script job"
```

Mpi 作业脚本内容参考：

```Bash
#!/bin/bash
#DSUB --mpi hmpi
#DSUB -q {your_queue}
#DSUB -n {your_job_name}
##DSUB -N 608
#DSUB -rpn 16
#DSUB -nn 1
#DSUB -oo out.%A
#DSUB -eo err.%A
export OMP_NUM_THREADS=36
np=16

EXE={your_exe}

source /work_ssd/software/HPCKit/25.2.1/setvars.sh  > /dev/null

RANK=0
for NODE in `awk '{print $1}' ${CCS_ALLOC_FILE} | sort`
do
    for NUMA in {0..15}
    do
        echo rank $RANK=$NODE slot=$(($NUMA * 38))-$(($NUMA * 38 +
35))
        RANK=$(($RANK + 1))
    done
done > $PWD/${CCS_JOB_ID}-rankfile

echo launch on $HOSTNAME

time \
mpirun \
    -x PATH -x LD_LIBRARY_PATH -x UCX_TLS=sm,rc \
    --mca rmaps_rank_file_physical true \
    --mca grpcomm_direct_priority 100 \
    --mca coll_tuned_use_dynamic_rules true \
    -x UCX_RNDV_THRESH=512K \
    -x OMP_PROC_BIND=close -x OMP_PLACES=cores\
    -x BINDPROCNIC_NOSHOW=1 \
    -x UCX_RC_VERBS_ROCE_LOCAL_SUBNET=y -x
UCX_UD_VERBS_ROCE_LOCAL_SUBNET=y \
    --mca plm_rsh_agent /usr/bin/ssh \
    -np  $((CCS_TASK_REPLICA))   --rankfile $PWD/${CCS_JOB_ID}-
```

```
rankfile  $EXE
```

```
参数说明
n  | --name     提交作业时，指定作业的名称。
-nl | --node-list 提交作业时，指定节点或资源池偏好。
-A | --account 提交作业时，提交作业到指定 Account 下。
-q | --queue  提交作业时，提交作业到指定队列下。
-N | --replica 提交作业时，指定作业的资源副本数量。
-o |  --output   指的输出文件名称。
-e | --error 提交作业时，重定向该作业的错误输出日志路径
-R | --resource 提交作业时，指定作业的资源需求。
-mR  | --min-resource  提交作业时，指定作业的最小资源需求。
--mpi 提交 MPI 作业时，指定作业的类型，Donau Scheduler 会根据指定的作业类型来调度执行 MPI 作业。
-rpn | --replica-per-node 提交作业时，指定作业在每个节点运行的最大任务数。
-nn  | --nnodes 提交作业时，指定作业运行的节点数。
```

3. 作业任务运行方式

核隔离机制

机制介绍

为减少操作系统后台任务对计算任务的影响，本集群使用 isolcpu 启动参数，启用了核隔离（CPU isolation）机制，集群中 CPU 核心分两类：

计算核心：

用途：专用于处理计算任务，不处理周期性时钟终端、不处理 RCU 回调等，最大限度减少系统噪声

核心列表： 0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606

系统核心：

用途：作为 housekeeping 系统核，处理系统服务、各系统 agent 进程、公共服务、处理中断等

核心列表：37,75,113,151,189,227,265,303,341,379,417,455,493,531,569,607（每个节点的 16 个 numa 中，每 numa 的最后一个核心）

在此配置下，命令行执行的进程会运行在系统核上，所以其只能感知到系统核范围内的 CPU 资源。如果想要将应用运行在计算核上，需要感知隔离核，必须显示的指定需要使用的计算核

任务拉起方式

taskset 方式

原本使用 takset 执行的作业，在核隔离场景下，需要注意-c 指定的 CPU 范围是否包含系统核。

禁止使用 taskset -c 0-607 来启动任务。

如需使用全部计算核，需要显示指定：taskset -c 0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606 ./your_app

numactl 方式

与 taskset 不同，numactl 自己没有在隔离核时，绑核范围也不能有隔离核。

可以使用以下几种方式解决（建议使用前两种）：

使用 taskset -c xx-xx 来替换；

在 numactl 前添加 taskset -c xx-xx；

numactl 在-C 参数前，增加“--all”参数；

示例：

taskset -c 0-31 numactl -C 0-31 ./your_app

taskset --all -C 0-31 ./your_app

rankfile 方式

rankfile 示例：

rank 0=cn10865 slot=0-36

rank 1=cn10865 slot=38-74

rank 2=cn10865 slot=76-112

rank 3=cn10865 slot=114-150

rank 4=cn10865 slot=152-188

rank 5=cn10865 slot=190-226

rank 6=cn10865 slot=228-264

rank 7=cn10865 slot=266-302

rank 8=cn10865 slot=304-340

rank 9=cn10865 slot=342-378

rank 10=cn10865 slot=380-416

rank 11=cn10865 slot=418-454

rank 12=cn10865 slot=456-492

rank 13=cn10865 slot=494-530

rank 14=cn10865 slot=532-568

rank 15=cn10865 slot=570-606

命令示例：

mpirun -np 4096 –rankfile rk16 --mca rmaps_rank_file_physical true -x PATH -x LD_LIBRARY_PATH ./your_app

作业任务运行方式（线程分布）确认参考

python

使用 threading.Thread，底层通过 pthread_create 实现

multiprocessing.Process（fork 模式），拉起子进程

使用 subprocess.Popen 或 os.system，执行新的可执行文件

C 语言

fork()

pthread_create()

vfork()

clone()

系统监测手段层面排查

运行核号确认

通过下列命令，确认各进程和线程所运行的核（第 9 列数据）：

ps -eLF | grep $your_app

系统调用确认

通过下列命令抓取所有的系统调用，在其中寻找关键字，如 clone 等：

拉起应用时：strace ./your_app

应用运行时：strace -p $app_pid

参考：

起进程时会抓到类似： clone(SIGCHLD, 0)

起线程时会抓到类似： clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD | ...)

FAQ

1. 作业运行后，均跑在系统核（37、75、113、151、189、…、607）上

问题根因：在核隔离环境中，命令行运行的默认核为系统核，在未正确指定使用计算核时，会出现该问题

解决方案：参考上述“作业任务拉起方式”中的描述，启动业务进程

2. 核隔离场景下 pthread 绑核提交示例

方式一：通过 LD_PRELOAD 环境变量，对当前进程及加载的动态库生效

```
LD_PRELOAD=/work_ssd/system/prehook/libpthread_hook.so taskset -c 0-36,38-74,76-112,114-150,152-188,190-226,228-264,266-302,304-340,342-378,380-416,418-454,456-492,494-530,532-568,570-606 ./your_app
```

方式二：修改源码；demo 代码如下，通过 pthread_setaffinity_np 进行绑核，使用类似方式调整源码：

```
#include <pthread.h>
#include <unistd.h>
#include <sched.h>

#define NUM_THREADS 32

// 线程函数：打印线程 ID 和运行的 CPU 核心
void* hello_thread(void* arg)
{
   int cpu = *(int*)arg;
   cpu_set_t set;
   CPU_ZERO(&set);
   CPU_SET(cpu, &set);
   pthread_setaffinity_np(
      pthread_self(),
      sizeof(set),
      &set);
   printf("thread %d cpu=%d\n",
        cpu,
        sched_getcpu());
   sleep(5);

   return NULL;
```

```
}
int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];
    printf("=== pthread Hello World (32 threads + CPU affinity) ===\n");
    printf("Total CPU cores: %d\n\n", (int)sysconf(_SC_NPROCESSORS_ONLN));
    // 创建 32 个线程
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i + 1;
        int rc = pthread_create(&threads[i], NULL, hello_thread, &thread_ids[i]);
        if (rc != 0) {
            fprintf(stderr, "Error: failed to create thread %d\n", i);
            return 1;
        }
    }
    printf("All 32 threads created. Waiting for completion...\n\n");
    // 等待所有线程结束
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("\nAll 32 threads finished. Main thread exiting.\n");
    return 0;
}
```

HMPI 使用参考

MPI UCX_TLS 参数

当进程规模大于等于 4096 节点 ppn 为 16 的场景下，使用 alltoall 等多对多集合通信时手动指定 UCX_TLS 为 rc 会超出 qp 硬件上限，需要手动指定为 sm,ud,或者不指定 TLS 参数（hmpi 会自动选择 TLS 协议）

集合通信算法参数

当前的默认算法可以支持大规模应用场景，如果想修改集合通信算法进行深度调优，可以通过参数进行选择使用 UCG 算法还是 tuned 算法。

使能 tuned 算法示例：

mpirun -np 4096 –rankfile rk16 --mca rmaps_rank_file_physical true -x PATH -x LD_LIBRARY_PATH \

--mca coll_tuned_use_dynamic_rules true --mca coll_tuned_allgather_algorithm 4 ./exe #选择 allgather 的算法 4

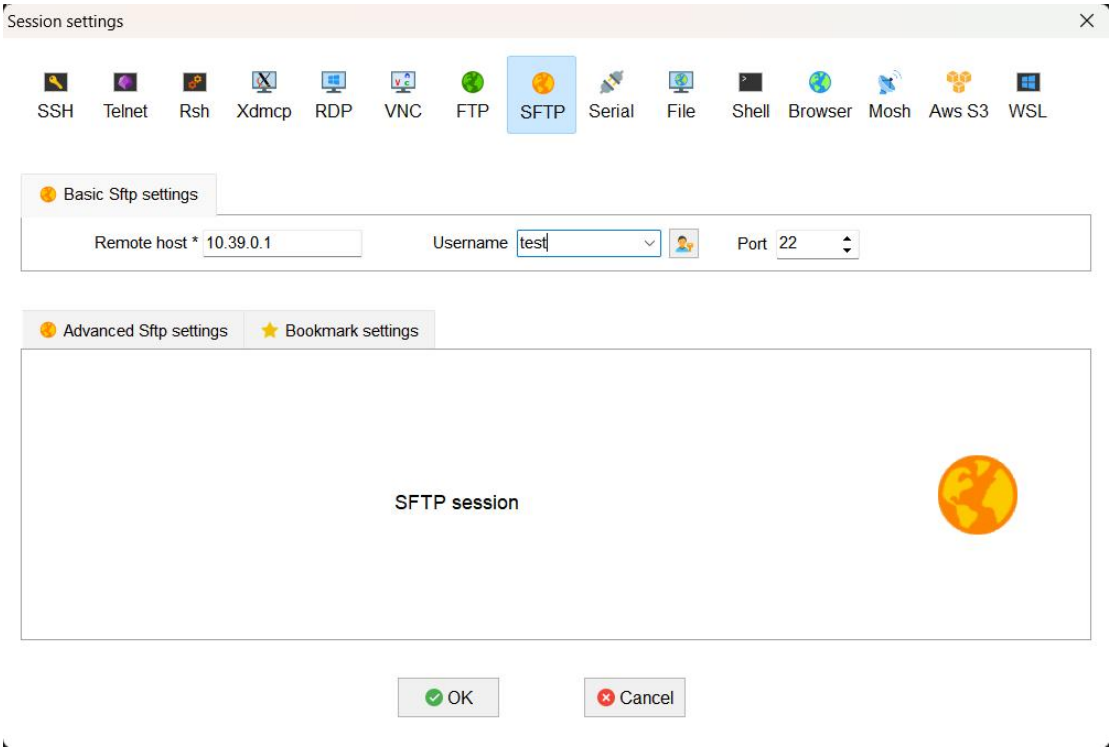
使能 ucg 算法算法示例：

mpirun -np 4096 –rankfile rk16 --mca rmaps_rank_file_physical true -x PATH -x LD_LIBRARY_PATH \

--mca coll_ucg_enable_coll gatherv -x UCG_PLANC_UCX_GATHERV_ATTR=I:4 ./exe #选择 gatherv 的算法 4

五、数据传输

1、可以通过 ftp 软件上传下载数据，协议 sftp，端口 22

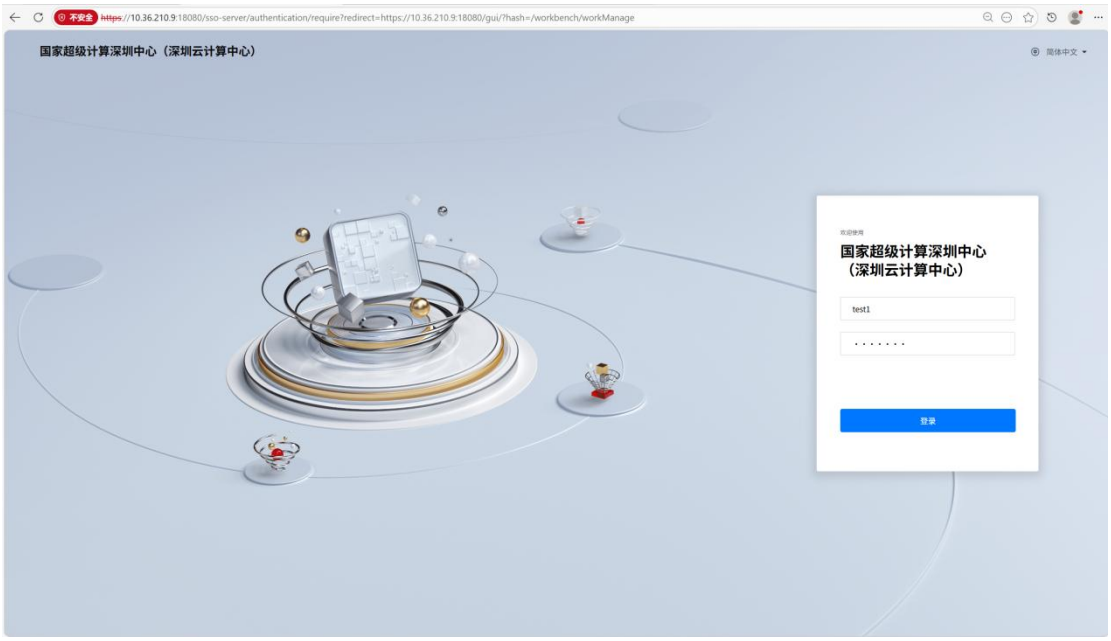


2、登录 Donau Portal 上传下载数据，详见 Web 页面登录使用。

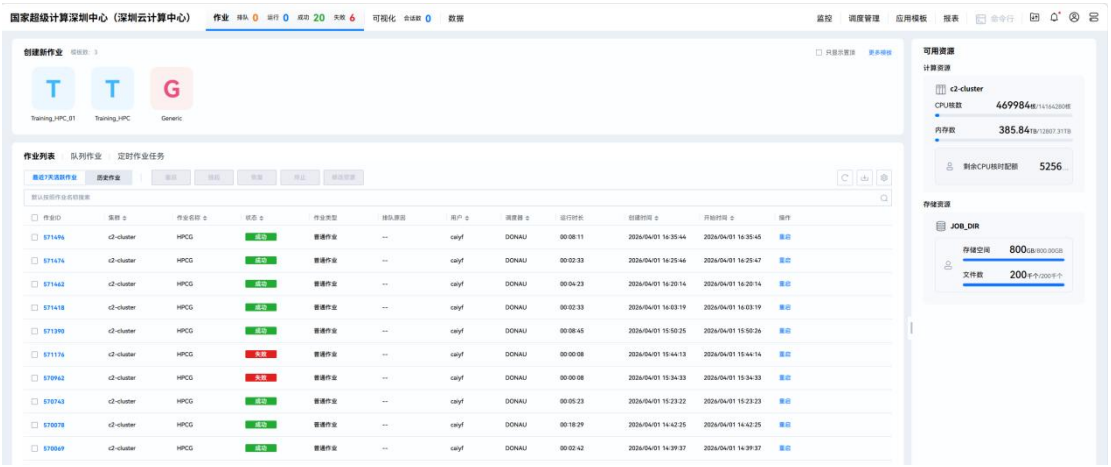
六、Web 页面登录使用

登录 Donau Portal。在 Web 浏览器地址栏中输入 Donau Portal 的 URL 地址 https://10.39.210.9:18080/gui

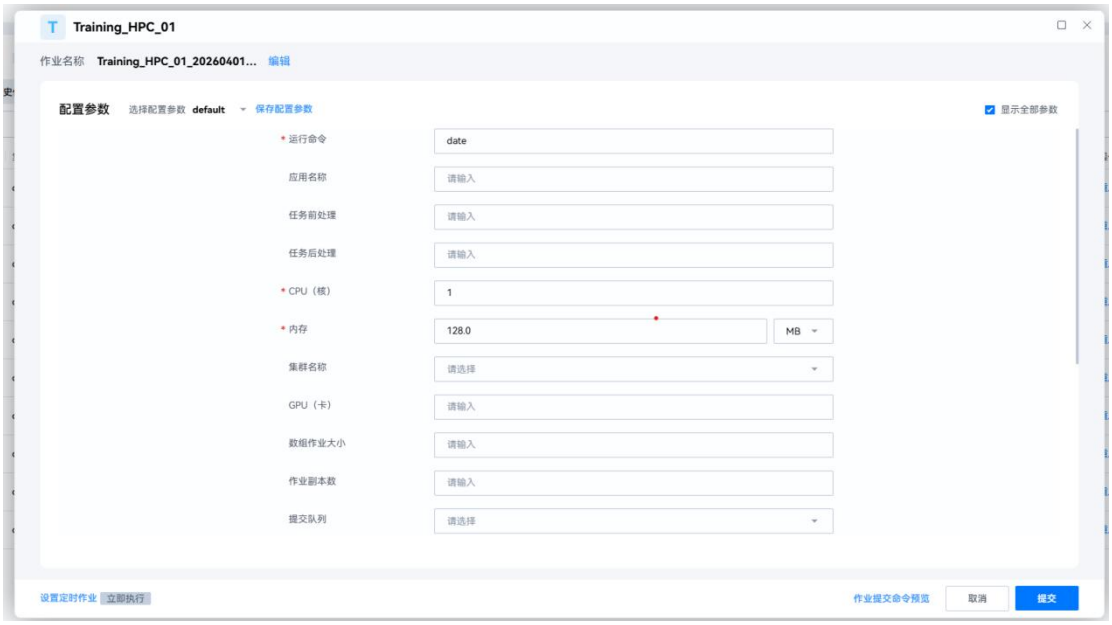
输入有效的用户名和密码。单击“登录”，进入 Donau Portal 系统界面。初次登录 Donau Portal，可以查看功能指引，了解各模块的功能和使用方式



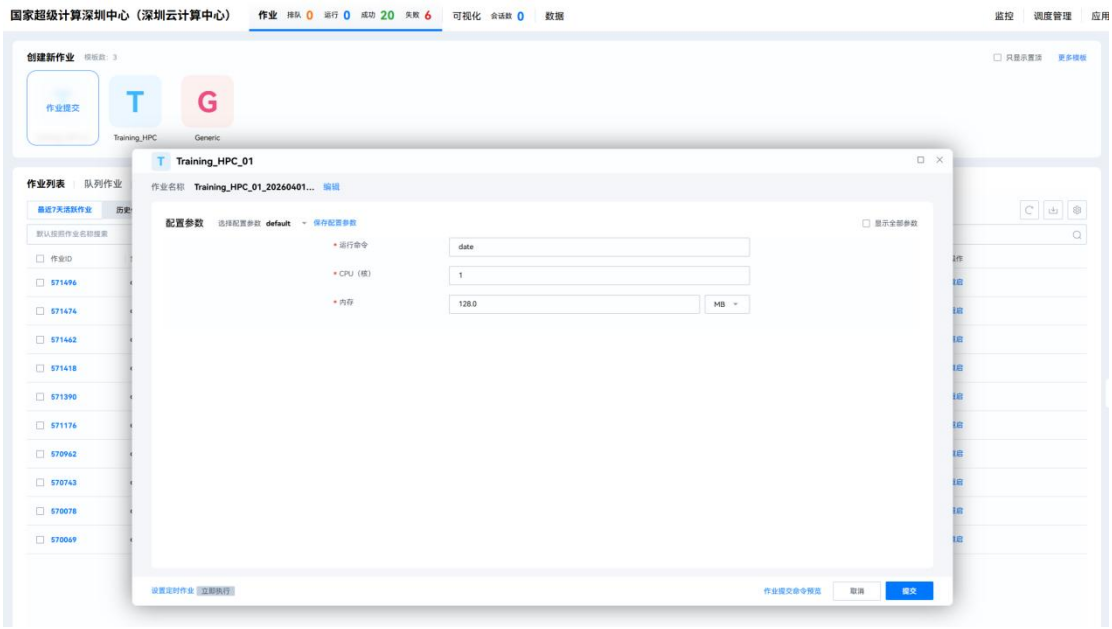
登录到 Donau Portal 界面，单击“作业”，右侧显示集群资源使用情况、账户剩余机时、用户存储配额和使用情况，下方显示作业列表，点击作业编号可以查看作业详细情况。



在“创建新作业”区域，选择需要提交的作业模板，进入提交作业界面配置作业参数。勾选“显示全部参数”，用户根据实际情况配置表单组件相关参数。参数填写完成后，可单击“保存配置参数”，输入配置参数名称，保存配置参数，便于下次进行快速填写。



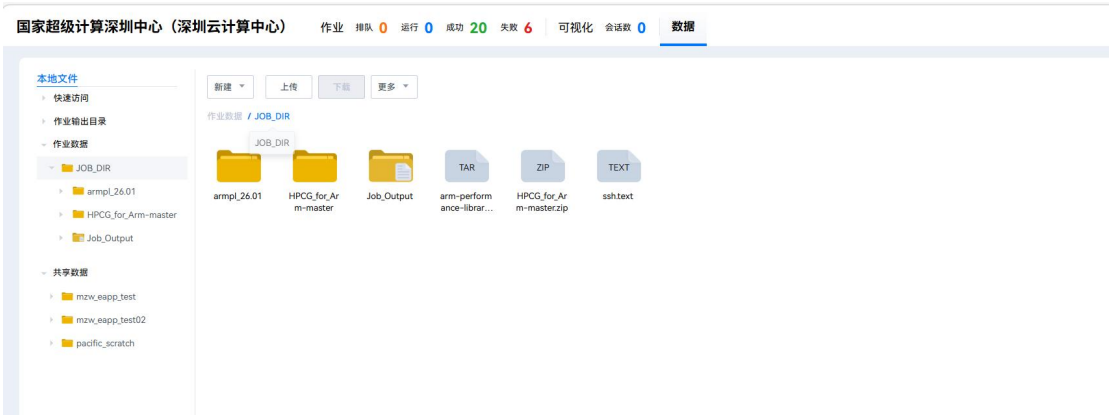
若之前保存过配置参数，可以不用勾选“显示全部参数”，直接快速完成参数填写。



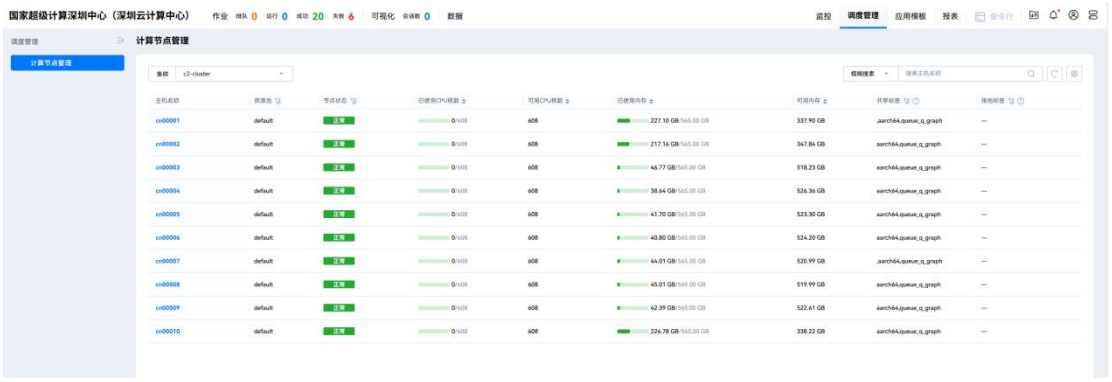
点击可视化选项可远程连接可视化应用



数据部分可以查看、创建、上传、下载文件。



调度管理可以查看可使用队列中计算节点的状态。



单击“应用模板”，进入应用模板界面。单击“作业模板”/“可视化模板”，进入作业模板/可视化模板界面。可查看已发布/未发布模板，并进行相关操作。



七、集群基础命令

1. 状态查询&作业管理

常用作业查询、提交、作业管理命令

查询队列内节点信息	dinfo -q your_queue
查询全部队列信息	dinfo -q all
查询指定节点的列表信息	dnode cn00002 dnode "cn00002 cn00003" dnode "cn[00002-00008,00010]"

作业提交-指定 prehhok	dsub --prehook "~/hook.sh" your_exe
作业提交-指定队列	dsub -q queuename your_exe
作业提交-指定目标节点	dsub -q queuename -nl 'cn[08961-09065]' your_exe
作业提交-指定标准输出和错误输出	dsub -q queuename -o ~/donau_job.log -e ~/donau_job.error your_exe
作业查看	djob jobid djob -l jobid
作业终止	dkill jobid
查看应用日志目录	djob -l jobid | grep RedirectOutputPath
访问正在运行作业的节点	dattach -l jobid | -en exec_nodeid

2. 调度脚本常用参数示例

脚本文件中#DSUB 为调度器选项的标识符；

如不需要可以使用两个#的方式进行注释：##DSUB，或直接删除该行

#DSUB --mpi hmpi	指定 MPI 类型为 Hyper MPI
#DSUB -x job	提交独占作业，独占作业运行的节点上不能同时执行其他作业
#DSUB -q {your_queue}	指定作业提交到的队列
#DSUB -n {your_job_name}	指定任务名称
#DSUB -N 608	用于指定作业运行需要的资源份数（一份资源是由-R 指定的多种资源的组合），默认一份资源会启动一个任务 如果没有指定该参数，且也没有指定 nn 和 rpn，则缺省值为 N=1 如果没有指定该参数，但是同时指定了 nn 和 rpn，则默认 N = nn(min) * rpn 如果没有指定该参数，但是仅指定了 rpn，则默认 N=rpn 且 nn=1 如果没有指定该参数，但是仅指定 nn，则默认 N=nn 且 rpn=1

#DSUB -rpn 16	用于指定作业在每个节点分配最大资源份数。如果作业的-N 资源副本数小于该值，则该作业会分配到一个节点上运行
#DSUB -nn 1	指定作业分布的节点数 支持指定最小、最大运行的节点数，标准用法为：-nn "min,max" 如果 min=max，可简化为：-nn x
#DSUB -R "cpu=4;mem=1G"	用于指定每份资源的具体资源需求；cpu 默认值为 1，mem 默认值为 128、默认单位 MB
#DSUB -oo out.%A	指定覆盖式重定向标准输出路径
#DSUB -eo err.%A	指定覆盖式重定向错误输出路径